Reviewer Pack — Minimal Rank of Ehrhart Semigroups Bounds Communication Comp…

Entry 6c975c602b79 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 09:51:31 UTC

Minimal Rank of Ehrhart Semigroups Bounds Communication Complexity for Symmetry Detection

Entry ID: `6c975c602b79` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 09:51:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Ehrhart Theory) **Field B** (complexity object): Communication Complexity: Symmetry Detection

Statement:

> {'sentence_1': 'For a given n-ary Boolean function f, the minimal rank of its Ehrhart semigroup, denoted as Rank_Ehrhart(f), is upper bounded by the maximum communication complexity for detecting symmetry in f, i.e., Rank_Ehrhart(f) ≤ CC_SymmetryDet(f).', 'sentence_2': 'Equivalently, for any function f with n variables, there exists a constant c(n) such that the number of distinct values that can be obtained by permuting arguments of f is at most c(n) times Rank_Ehrhart(f).', 'sentence_3': 'For all instances of size n ≤ 40, if the communication complexity for symmetry detection exceeds Rank_Ehrhart(f), then there exists a permutation-invariant set of inputs that cannot be distinguished from a uniformly random set using polynomially many queries.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Ehrhart theory studies the combinatorial properties of lattice points in polyhedral regions, which can be seen as a measure of complexity for counting problems. The minimal rank of an Ehrhart semigroup is a specific invariant that captures the complexity of this counting process.', 'sentence_2': 'Communication complexity for symmetry detection measures how much communication is needed to distinguish whether a given function exhibits some form of symmetry or not. By linking these two concepts, we may uncover new insights into the computational hardness of detecting symmetries in Boolean functions.', 'sentence_3': 'If the conjecture holds, it would provide a novel method for analyzing and bounding communication complexity problems using algebraic combinatorics.'}

Taxonomy category: `ERHART_COMMUNICATION_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7d065b329e85e18a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture that minimal rank of Ehrhart semigroups bounds communication complexity for symmetry detection, support is established if at least 80% of the 30 test functions show a correlation where Rank_Ehrhart(f) ≤ CC_SymmetryDet(f), and no instance exceeds this bound by more than 3 standard deviations from the mean. Falsification occurs if any seed produces an instance with CC_SymmetryDet(f) > Rank_Ehrhart(f) by more than 3 standard deviations from the mean.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- "Ehrhart semigroup" AND "communication complexity" AND symmetry detection" - "algebraic combinatorics" AND "maximal communication complexity" AND "symmetry detection" - "Rank_Ehrhart(f)" AND c(n) AND "distinct values permuting arguments"`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = int(math.log2(len(f)))
        max_queries = 2**n
        queries = set()
        while len(queries) < max_queries:
            query = tuple(random.sample(range(n), n))
            if f[query] not in [0, 1]:
                return -1
            queries.add(query)
        return len(queries)

    def ehrhart_semigroup(f):
```

```
    n = int(math.log2(len(f)))
    semigroup = set()
    for i in range(2**n):
        binary = format(i, f'0{n}b')
        count = sum(int(bit) for bit in binary)
        if count % 2 == 0:
            semigroup.add(count // 2)
    return len(semigroup)

def rank_ehrhart(f):
    n = int(math.log2(len(f)))
    semigroup = ehrhart_semigroup(f)
    if semigroup == 1:
        return 1
    for r in range(2, n + 1):
        matrix = [[0] * (r + 1) for _ in range(r + 1)]
        for i in range(r + 1):
            matrix[i][i] = 1
        for j in range(r):
            for k in range(j + 1, r + 1):
                matrix[j][k] = -matrix[k][j]
        det = gaussian_elimination(matrix)
        if det == 0:
            return r
    return n

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        pivot_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[pivot_row][i]):
                pivot_row = j
        A[i], A[pivot_row] = A[pivot_row], A[i]
        if A[i][i] == 0:
            return 0
        for j in range(i + 1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] += factor * A[i][k]
    det = 1
    for i in range(n):
        det *= A[i][i]
    return det

def is_permutation_invariant(f, perm):
    n = int(math.log2(len(f)))
    for i in range(2**n):
        binary = format(i, f'0{n}b')
        permuted_binary = ''.join(binary[perm[j]] for j in range(n))
        if f[i] != f[int(permuted_binary, 2)]:
            return False
    return True

def count_distinct_values(f):
    n = int(math.log2(len(f)))
```

```
        distinct_values = set()
        for i in range(2**n):
            binary = format(i, f'0{n}b')
            permuted_binary = ''.join(binary[perm[j]] for j in range(n))
            distinct_values.add(f[int(permuted_binary, 2)])
        return len(distinct_values)

    def find_counterexample(f):
        n = int(math.log2(len(f)))
        perms = list(itertools.permutations(range(n)))
        for perm in perms:
            if not is_permutation_invariant(f, perm):
                return f"Permutation {perm} does not preserve symmetry"
        return ""

    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    cc_symmetry_det = communication_complexity(f)
    rank_ehrhart_f = rank_ehrhart(f)
    distinct_values = count_distinct_values(f)
    counterexample = find_counterexample(f)

    if cc_symmetry_det > rank_ehrhart_f:
        return {
            "metric_name": "Rank_Ehrhart vs CC_SymmetryDet",
            "metric_value": rank_ehrhart_f,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": counterexample
        }
    else:
        return {
            "metric_name": "Rank_Ehrhart vs CC_SymmetryDet",
            "metric_value": rank_ehrhart_f,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std_dev = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
```

```
elif any(r["counterexample"] != "" for r in results):
    first_failing_seed = next((r["seed"] for r in results if r["counterexample"] != ""), None)
    print(f"RESULT: FALSIFIED counterexample=\"{next(r['counterexample'] for r in results if r
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code timed out before producing data, which means it did not complete within the expected time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13953
2	preregistration	ollama_remote	glm4:latest	0	0	6035
3	novelty	ollama_remote	glm4:latest	0	0	4914
4	novelty	ollama_remote	glm4:latest	0	0	5742
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18159
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10643
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10056
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15419
9	judge	ollama_remote	glm4:latest	0	0	26986

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111907 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/6c975c602b79.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/6c975c602b79.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6c975c602b79.tar.gz` (if generated)